

SQL

CREATE DATABASE - creates new database

CREATE DATABASE db_name;

CREATE DATABASE IF NOT EXISTS db_name;

DROP DATABASE - deletes entire database permanently

DROP DATABASE db_name;

SHOW DATABASE - displays every database

SHOW DATABASES;

SHOW TABLES - displays every table inside current database

SHOW TABLES;

USE - selects which database to work in

USE db_name;

Create table - creates a new table

CREATE TABLE table-name (
column data-type constraint,
column data-type constraint,

Insert into - Adds new rows into table

INSERT INTO table-name (column1, column2)
VALUES (value1, value2)

char(n) - stores fixed length string

varchar(n) - stores a variable length string

TINYINT - stores very small whole numbers

command types:

DDL - defines or changes database structure

DML - removes data

DML - changes data inside tables

DCL - controls user permission

DDL - controls transactions

primary key - uniquely identifies every row
foreign key - links one table to another

Constraints:

NOT NULL - prevents null values : name VARCHAR(50) NOT NULL

UNIQUE - prevents duplicate values : email VARCHAR(100) UNIQUE

primary key - makes values unique + not null : id INT primary key

foreign key - ensures matching values : foreign key (student id) references (teacher id)

DEFAULT - will give a default value if none entered : country DEFAULT 'UK'

CHECK - restricts allowed values : age INT CHECK (age > 16)

WHERE - filters rows using a condition

SELECT *
FROM TABLE

WHERE condition;

limit - limits how many rows can be returned
LIMIT 5

Order by - sorts results

Select - From

ORDER BY column ASC/DISC

Aggregate functions - used in the select statement or condition
Count(), Max(), Min(), Sum(), Avg()

Group By - groups rows with the same values

Select - From

Group By column;

HAVING - filters grouped data

Select - From

Group By column;

HAVING condition;

UPDATE - changes existing data

UPDATE table

Set column = value

WHERE condition;

DELETE - deletes selected rows

DELETE FROM students

WHERE age < 18

ON DELETE CASCADE - deletes all related rows on which table is deleted in which

Foreign key (id)

References table (id)

ON DELETE CASCADE;

ON UPDATE CASCADE;

ON UPDATE CASCADE - updates all related rows on child table if updated on adult

ALTER - adds new column

ALTER TABLE students

ADD phone VARCHAR(40);

DROP column - delete a column

ALTER TABLE students

DROP column phone;

RENAME TO - renames a table

ALTER TABLE students

RENAME TO pupils;

Change column - renames a column add / or delete

ALTER table students

CHANGE column age student-age INT;

MODIFY - changes datatype or constraints

ALTER table students

MODIFY age INT;

Truncate - deletes contents of table but keeps table

TRUNCATE TABLE table-name;

INNER JOIN - Returns matching rows from both tables

LEFT JOIN - returns all left rows + matching right rows

RIGHT JOIN - returns all right rows + matching left rows

FULL OUTER JOIN - returns every row from both tables

JOIN type tables

ON table1, column1 = table2, column

CROSS JOIN - returns every possible combination

CROSS JOIN table1,

Self join - joins a table to itself

SELECT *

FROM table1, t1

JOIN table1 ON t1

ON t1, column1 = t2, column1

UNION - combines results + removes duplicates

SELECT name FROM students

UNION / ALL

SELECT name FROM teachers;

UNION ALL - combines results + keeps duplicates

Subquery - runs a query inside another

SELECT column

FROM table

WHERE column > C

SELECT Avg(column)

FROM table;

Views - Save query as a virtual table

CREATE VIEW student_view AS

SELECT * FROM ...

Stored procedures - Since SQL code that can be executed later

Create procedure proc1

As

Select * From students;

- When you want to call it - EXEC proc

Distinct - removes duplicate rows

Select Distinct city

Coalesce - replaces null with another value

SELECT

Coalesce (phone, 'no phone')

From students;

Triggers - Automatically runs SQL after or before an event

CREATE TRIGGER trigger_name

AFTER INSERT

ON students

FOR EACH ROW

BEGIN ...

END

LIKE - searches for text patterns

Select * From ...

WHERE name LIKE 'S%';

!= Any number of characters

= exactly one character

OVER - performs calculations without grouping rows

SELECT Avg (salary)

OVER (partition by department)

FROM employees;

Row number () - gives every row a unique number

Row number () OVER (ordered by salary DESC)

RANK - gives tied values the same rank with gaps

Dense Rank - gives tied values the same rank without gaps

DENSE/RANK() over (ordered by salary DESC)

LEAD - looks at next row

Lag - looks at previous row

LEAD /LAG (salary)

OVER (order by salary);

WITH - creates a temporary result set for one query

With high-scores AS

SELECT *

FROM grades

WHERE marks > 80)

call it - SELECT *

FROM high-scores